Refuelling your Jetpack

Thomas Künneth

MATERNA
Virtual Solution

Experts

What is Android Jetpack?

- Suite of **libraries**, **tools**, and **guidance** for less boilerplate and a clearer structure
- New features without waiting for users to upgrade their devices to new Android versions
- Instead of ***Do it however you want***, Google has an **opinion** about how things should be done
- Backward compatibility built in (fragmentation and OS differences are handled for you)

The Era of Fragmentation

- New features used to be tied to OS updates, but OS updates took ages to reach devices
- The Android Support Library (`android.support.*`) was Google's band-aid. It sort of helped, but…
- …it turned into a mess. `v4`, `v7`, `v13`… jar files everywhere
- A dependency hell

The Reboot – Enter Android Jetpack

- 2017: Architecture Components (*ViewModel*, *Room*, *Lifecycle*, *LiveData*) announced at I/O; 1.0 that November
- 2018: Jetpack and the great reboot to `androidx.*`
  - libraries unbundled from the OS
  - Own release cycles
  - Semantic versioning

Google got opinionated about architecture

From
Do it however it works for you
to
Here's what we think works best

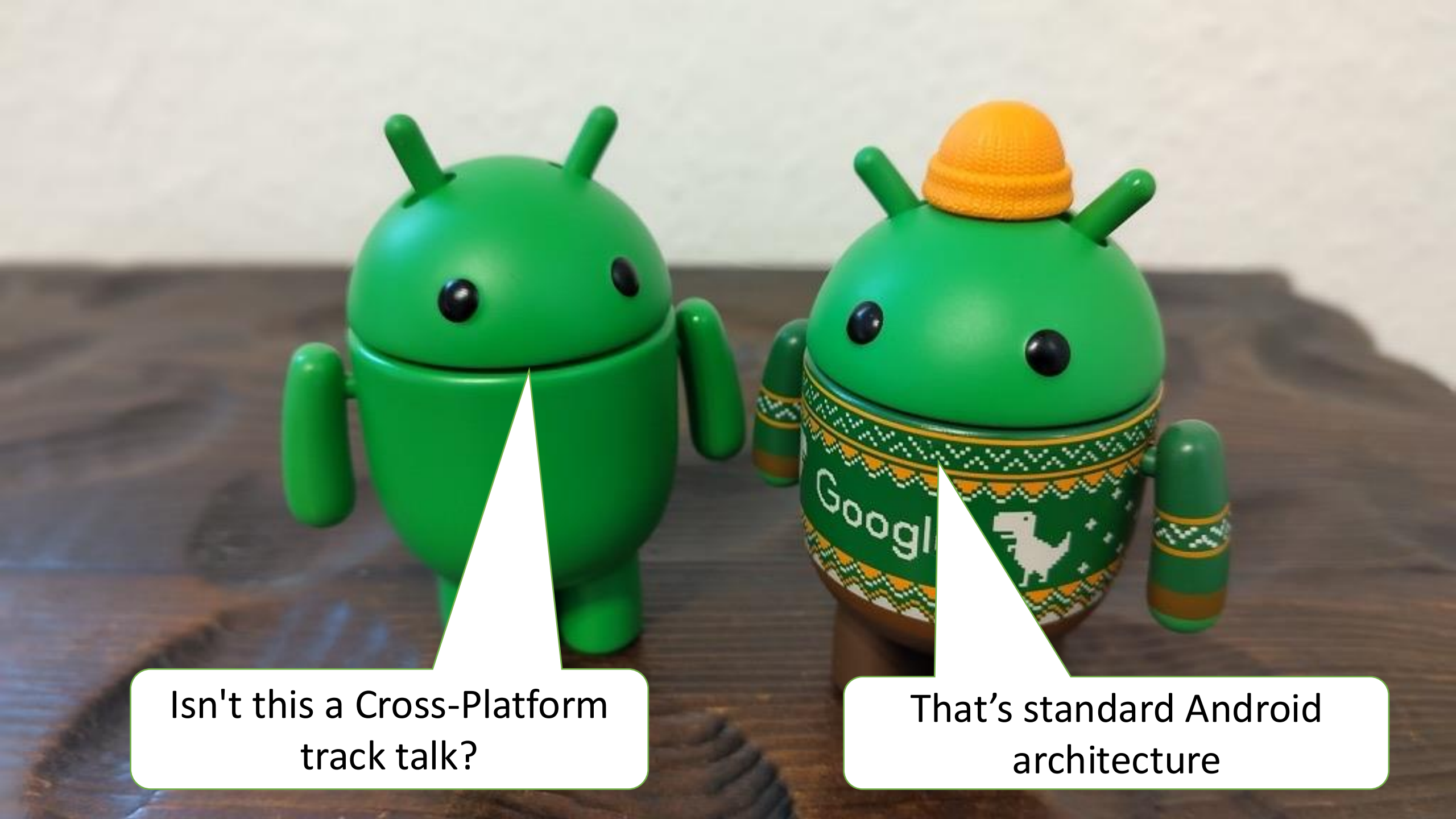
MATERNA
Virtual Solution

Fast forward to the Jetpack Jungle

- Today there are 130+ artifacts
- Not just modularity; instead, it's baggage
- Zombie libraries: deprecated, duplicate, still haunting the docs e.g. `lifecycle-extensions` (deprecated, split into separate `lifecycle-*` libs), `security-crypto` (deprecated, no clear successor)
- How to build a modern architecture with three ways to do the same thing?

The Official Architecture

- Google's answer to the chaos: a curated stack (the Golden Path)
- UI reacts to state, events flow up. ViewModels hold state and survive config changes
- One navigation graph. One back stack. No reimplementing per screen
- Reactive, typed persistence: Room for the DB, DataStore for preferences / configuration

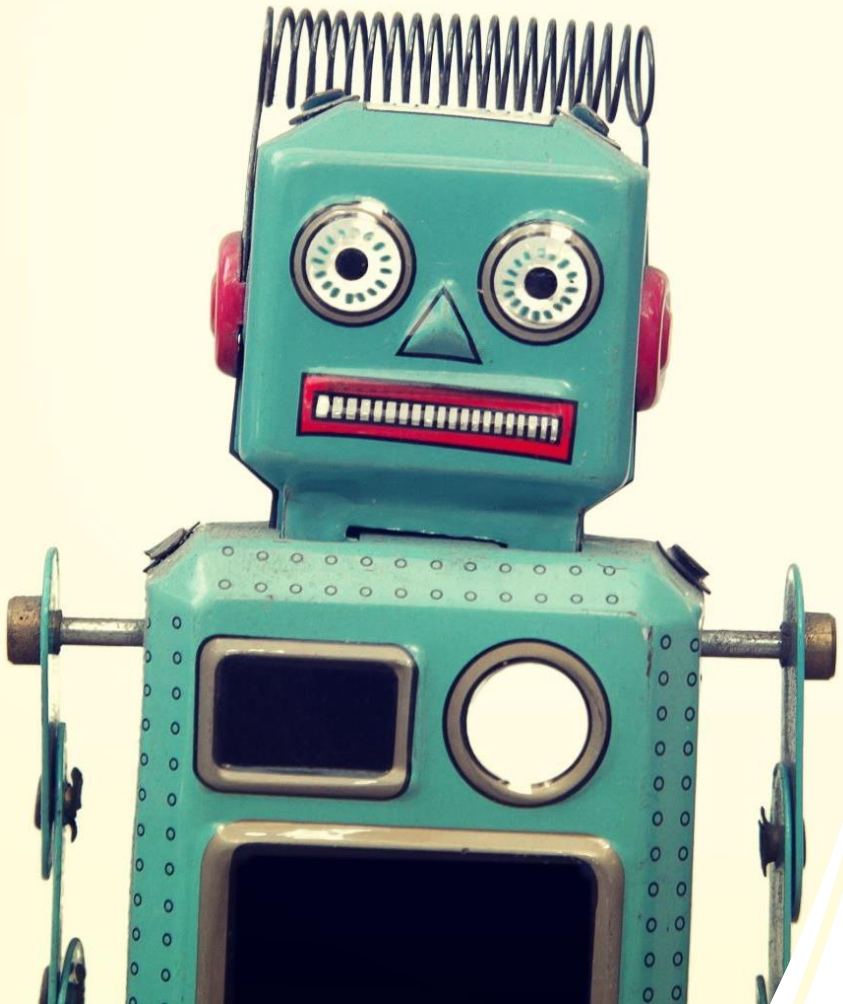

Isn't this a Cross-Platform track talk?
That’s standard Android architecture

The Twist

- That stack you just saw runs on iOS, Desktop, and Web
- Same APIs
- One architecture, the same code

Compose Multiplatform

- Same Jetpack Compose that every Android dev loves, just unchained from Android
- Declarative, reactive: one mental model for every screen
- On iOS and Desktop, it uses Skia and runs natively
- No more learning three UI frameworks
- Android skills deploy to iOS, Desktop, and Web



State & Lifecycle: (finally) adopting the Standard

- Android didn't invent the ViewModel; others had it years before
- An idea too good to neglect
- You put ViewModels in `commonMain`
- They survive config changes (e. g. rotation) and backgrounding on all platforms

Experts

Navigation

- With Navigation Compose you typesafe-ly define your graph and routes once in `commonMain`
- Back stack and deep links work the same on Android, iOS, and Desktop
- Activities, View Controllers, and windows are just thin hosts
- Instead, shared code drives the UI

MATERNA
Virtual Solution

Persistence: Room

- *Room* is an ORM on SQLite: Kotlin classes are mapped to tables, DAOs, Flow for reactive queries
- Compile-time SQL verification: no nasty runtime surprises. Runs on each platform's native SQLite
- One database layer in shared code
- ViewModels or repositories consume it the same way everywhere

Configuration & Preferences: DataStore

- *DataStore* allows typed, async reading / writing of preferences and configuration files
- Not blocking the UI thread
- Choose between key-value or protocol buffers. Same API on every platform. Settings, feature flags, preferences—all in shared code
- No more `UserDefaults` for iOS and `SharedPreferences` for Android

One architecture, (at least) three platforms

- The shared module holds almost everything: UI, ViewModels, navigation, Room, DataStore
- Platform apps are thin shells: they just bootstrap

Experts

The *shared* module structure

- `commonMain` contains Compose UI, ViewModels, Navigation, Room, DataStore
- When you need something platform-specific (file paths, open URL, back button), you use `expect` (interface) / `actual` (implementation)

Platform Entry Points

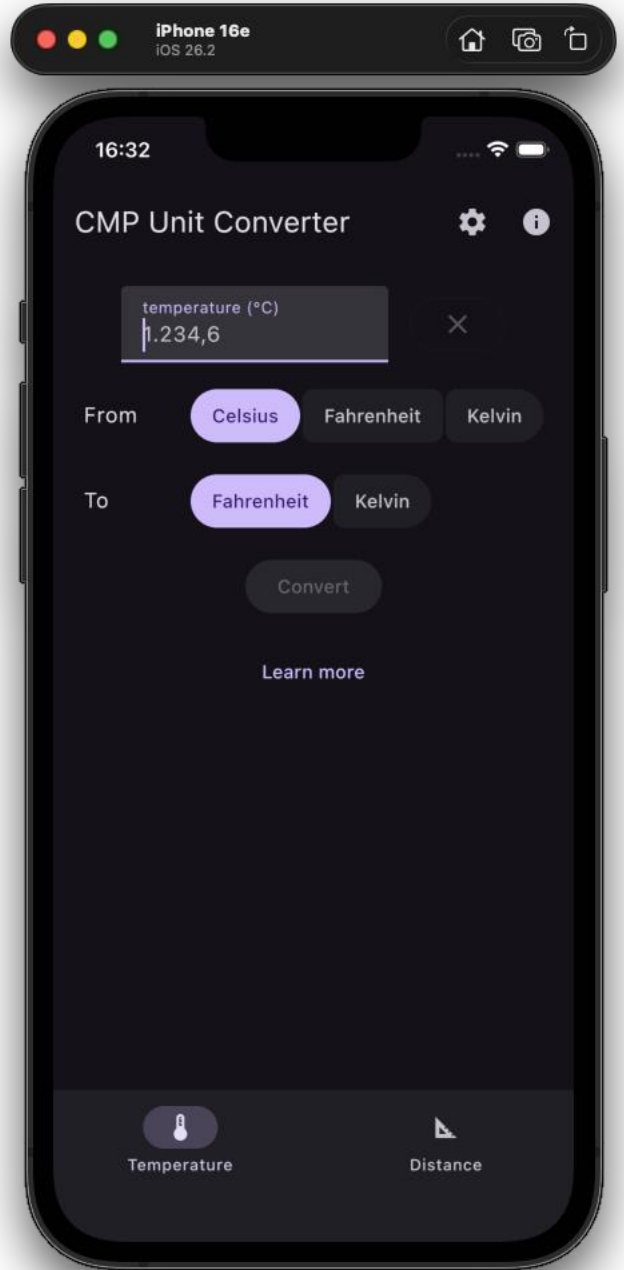
- Android: `Activity` and `setContent`
- iOS: a view controller hosting Compose
- Desktop: `main()` and one or more windows
- Each one wires lifecycle and hosts the same root composable
- You initialize your dependency injection framework at startup; the rest is shared

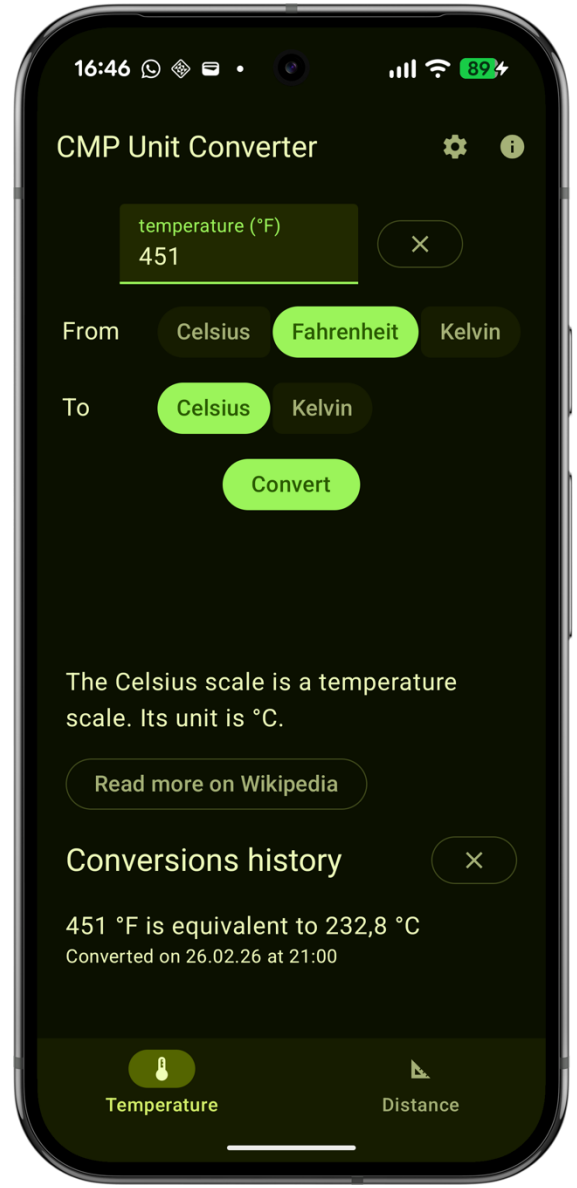
Dependency Injection: The Hilt Question

- Hilt is the recommended DI on Android. But Hilt is Dagger and annotation processing
- It doesn't run on iOS or Desktop
- Google hasn't announced a multiplatform roadmap for Hilt. For shared code today, we use a pure Kotlin option (for example, Koin)
- You define modules in shared code; platform supplies the actuals (e.g. where's the data directory). One container, initialized once per platform

CMP Unit Converter

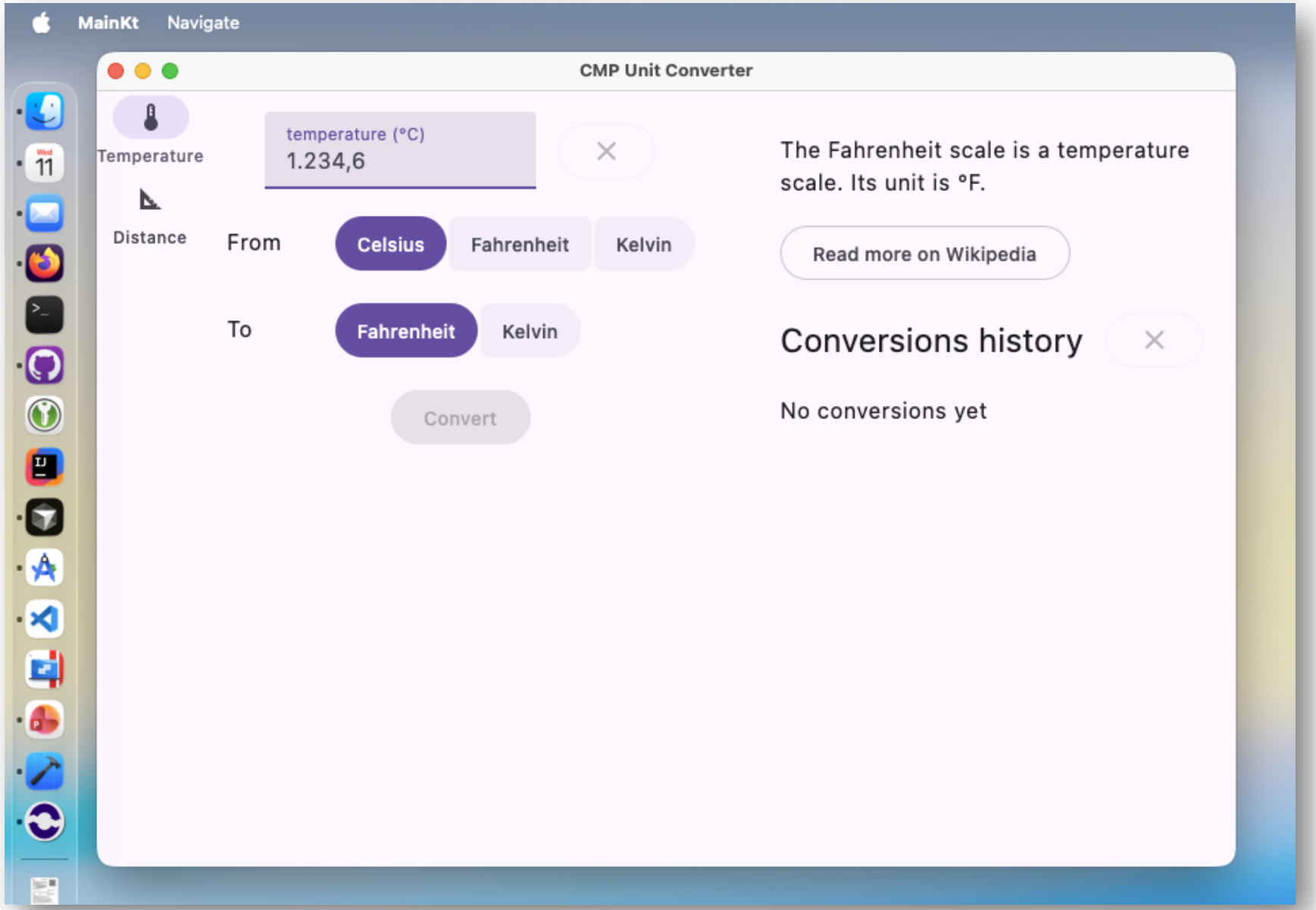
- A real app: unit conversion (temperature, distance, etc.) on Android, iOS, and Desktop
- Built on the full *golden path*: Compose, ViewModel, Room, DataStore, Koin
- One codebase, minimal platform glue





MATERNA
Virtual Solution

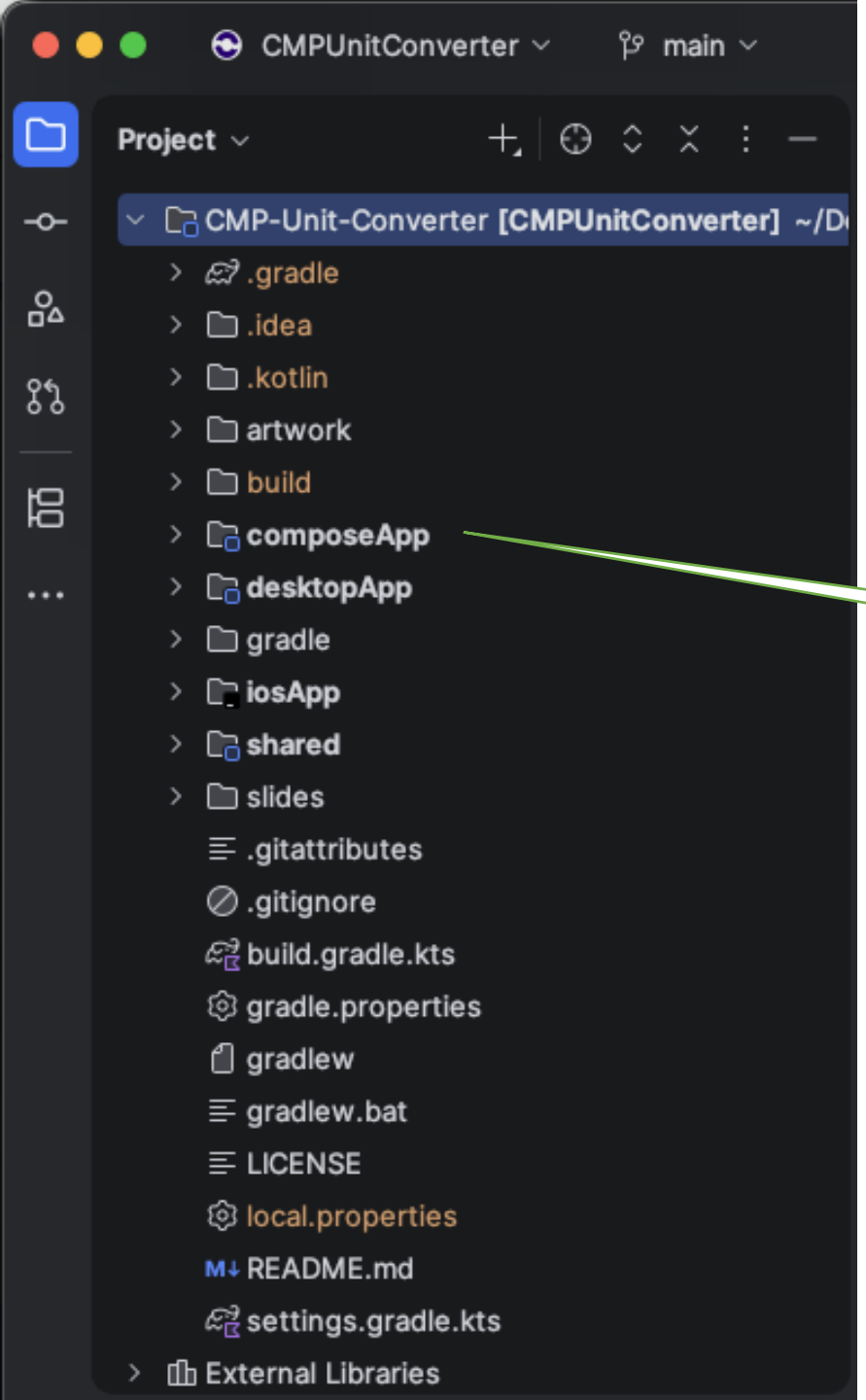
Experts



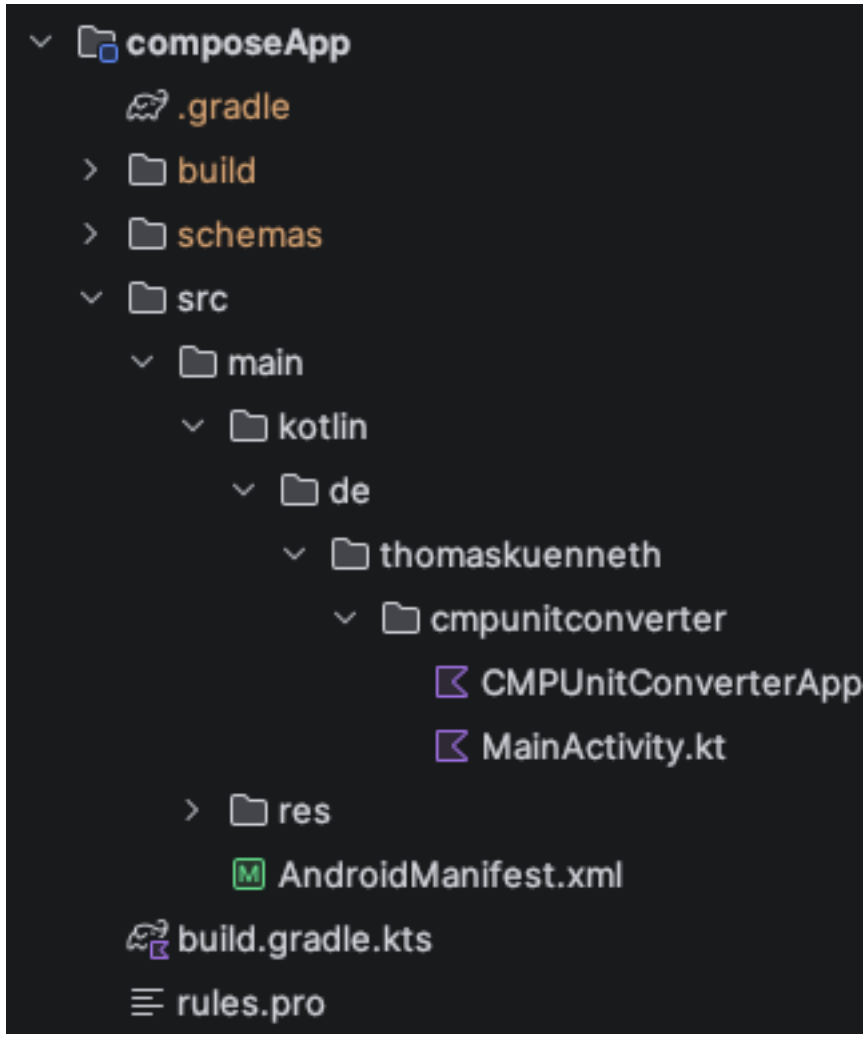
MATERNA
Virtual Solution

Experts

https://github.com/tkuenneth/CMP-Unit-Converter



Android



Experts

CMPUnitConverter main

Project

- CMP-Unit-Converter [CMPUnitConverter] ~/D
  - .gradle
  - .idea
  - .kotlin
  - artwork
  - build
  - composeApp
  - desktopApp
  - gradle
  - iosApp
  - shared
  - slides
  - .gitattributes
  - .gitignore
  - build.gradle.kts
  - gradle.properties
  - gradlew
  - gradlew.bat
  - LICENSE
  - local.properties
  - README.md
  - settings.gradle.kts
- External Libraries

Desktop

- desktopApp
  - build
  - src
    - main
      - kotlin
        - de.thomaskuenneth.cmpunitcor
          - AboutWindow.kt
          - main.kt
          - SettingsWindow.kt
  - build.gradle.kts
  - rules.pro

Experts

CMPUnitConverter main

Project

- CMP-Unit-Converter [CMPUnitConverter] ~/D
  - .gradle
  - .idea
  - .kotlin
  - artwork
  - build
  - composeApp
  - desktopApp
  - gradle
  - iosApp
  - shared
  - slides
  - .gitattributes
  - .gitignore
  - build.gradle.kts
  - gradle.properties
  - gradlew
  - gradlew.bat
  - LICENSE
  - local.properties
  - README.md
  - settings.gradle.kts
- External Libraries

iOS

- iosApp
  - iosApp ~/Development/Personal/Desktop
    - Configuration
    - Frameworks
    - iosApp
    - Products
  - Frameworks

Experts

CMPUnitConverter main

Project

- CMP-Unit-Converter [CMPUnitConverter] ~/D
  - .gradle
  - .idea
  - .kotlin
  - artwork
  - build
  - composeApp
  - desktopApp
  - gradle
  - iosApp
  - shared
  - slides
  - .gitattributes
  - .gitignore
  - build.gradle.kts
  - gradle.properties
  - gradlew
  - gradlew.bat
  - LICENSE
  - local.properties
  - README.md
  - settings.gradle.kts
- External Libraries

shared

- shared
  - build
  - schemas
  - src
    - androidMain
    - commonMain
      - composeResources
        - drawable
        - values
        - values-de-rDE
      - kotlin
        - de
          - thomaskuenneth
            - cmpunitconverter
    - desktopMain
    - iosMain
  - build.gradle.kts
  - rules.pro

Experts

Shared UI & Resources

- Composables live in `commonMain`. One UI implementation for all three platforms
- Icons and strings go in `composeResources`. You reference them with the generated `Res` API
- For example, `Res.string.app_name`, `Res.drawable.ic_thermostat`

```
@Composable
fun About(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier.padding(8.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(Res.drawable.artwork_no_background), null,
            modifier = Modifier.size(96.dp)
        )
        Text(
            text = stringResource(Res.string.app_name),
            modifier = Modifier.padding(top = 16.dp),
            color = MaterialTheme.colorScheme.onSurface
        )
        Text(
            text = "$VERSION ($BUILD_NUMBER)",
            color = MaterialTheme.colorScheme.onSurface
        )
        Text(
            text = platformName,
            style = MaterialTheme.typography.bodySmall,
            textAlign = TextAlign.Center,
            color = MaterialTheme.colorScheme.onSurface
        )
    }
}
```

ViewModel & Koin

- The converter ViewModels (e.g. `TemperatureViewModel`, `DistanceViewModel`) hold the math and state
- Defined in a *Koin* module in `commonMain`
- The UI gets them via `koinViewModel()`
- They don't know (or care) if they're running on an iPhone or a Pixel

Android

- appleMain
- appleTest
- commonMain
  - kotlin
    - de.thomaskuenneth.cmpunitconverter
      - app
      - composables
      - di
        - AppModule.kt
        - KoinApp.kt
      - distance
        - DistanceRepository.kt
        - DistanceSupportingPaneUseCase
        - DistanceViewModel
      - temperature
      - AbstractBaseRepository
      - AbstractConverterViewModel
      - AbstractSupportingPaneUseCase
      - AppDestinations
      - AppIcons

AppModule.kt

```kotlin
package de.thomaskuenneth.cmpunitconverter.di

import ...

1 Usage
val appModule = module {
    single<AppDatabase> { getRoomDatabase( builder = getDatabaseBuilder()) }
    single<HistoryDao> { get<AppDatabase>().getHistoryDao() }

    singleOf( constructor = ::HistoryRepository)

    singleOf( constructor = ::AppRepository)
    viewModelOf( constructor = ::AppViewModel)

    singleOf( constructor = ::TemperatureRepository)
    singleOf( constructor = ::TemperatureSupportingPaneUseCase)
    viewModelOf( constructor = ::TemperatureViewModel)

    singleOf( constructor = ::DistanceRepository)
    singleOf( constructor = ::DistanceSupportingPaneUseCase)
    viewModelOf( constructor = ::DistanceViewModel)
}
```

Platform Entry Points

- Android: Application calls `initKoin` at startup; `MainActivity` calls `setContent`. iOS: `ComposeView` inside a `ViewController`. Desktop: `main()` launches the window
- One quirk: Kotlin/Native exports `Unit`-returning top-level functions with a `do` prefix in Swift
- So, you have `initKoin()` in Kotlin but call `doInitKoin()` from Swift

```swift
import SwiftUI
import ComposeApp

@main
struct iOSApp: App {

    init() {
        // Kotlin/Native exports Unit-returning top-level functions with a "do" prefix in Swift/Obj-C.
        // Shared code has initKoin(); the generated API is doInitKoin(). See Koin CMP quickstart:
        // https://insert-koin.io/docs/quickstart/cmp
        KoinAppKt.doInitKoin()
    }

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}
```

Lessons from the trenches

- Keep Compose Multiplatform, Compose Compiler plugin, and Kotlin Multiplatform plugin in sync. The docs have tables—use them
- Your Android app's `activity-compose` must match the Compose version the shared module uses. If they're out of sync, you can get `NoSuchMethodError` at `setContent`, and other runtime surprises
- When you upgrade, check the compatibility page first. It saves a lot of debugging

- When starting a new project, use the AGP 9 structure from day one
- Migrating later works but is quite some work
- Put platform quirks behind `expect`/`actual` and document them (like `doInitKoin`)
- Keeps shared code clean and stops people from fixing what isn't broken

Time to wrap things up: what I showed you

- **The story**: Jetpack history, the Jungle, the golden path, and the twist—same stack, beyond Android
- **The lineup**: *Compose Multiplatform*, *ViewModel*, *Navigation*, *Room*, *DataStore*. All in KMP
- **How it fits**: one shared module, thin platform apps, entry points, DI
- **The case study**: CMP Unit Converter—one codebase, Android, iOS, Desktop. Clone it and try it

What I left out

- **Testing**: you can unit test ViewModels, repositories, and use cases in `commonTest`. Room: in-memory DB, same DAOs. expect/actual: test doubles. The docs and the sample repo have the details
- **Version alignment**: (we had a slide on it) `activity-compose` and Compose versions must match. When you upgrade, check the compatibility tables
- **What's not (yet) in KMP**: WorkManager, CameraX, and some others are still Android-only. For those you keep them in the app module or find alternatives

Jetpack roadmap beyond Android

- Google and JetBrains did a great job in unbundling Jetpack from the Android platform
- More libraries will get KMP support. The golden path stack is already stable and production-ready
- One architecture, many platforms—that's the direction. Investing in this stack isn't a bet on a niche

Thank you!

@tkuenneth

@tkuenneth

@tkuenneth@mastodon.social

https://github.com/tkuenneth/CMP-Unit-Converter

MATERNA Virtual Solution

Experts

Feedback? Yes, please

Refuelling your Jetpack - using Jetpack libraries beyond Android

MATERNA
Virtual Solution

Experts